



MedSim Pro

Medical Training Simulator Platform

Документ: **Техническое Задание (ТЗ)**

Версия: **1.0**

Дата: **Июнь 2026**

Составил: **Senior Full-Stack Developer**

Целевые модели: **GLM-5.1 / DeepSeek V4 / Kimi K2.6 / OpenCode**

Статус: **ГОТОВО К РАЗРАБОТКЕ**



01

Общая информация о проекте

MedSim Pro — это веб-платформа для медицинского образования, объединяющая интерактивный 3D-симулятор пациента с системой клинических упражнений. Студенты-медики практикуют диагностику в реалистичной среде без риска для реальных пациентов.

Название проекта	MedSim Pro — Medical Training Simulator
Тип	Web-платформа (SPA + серверная часть)
Целевая аудитория	Студенты медицинских вузов, интерны, ординаторы
Язык интерфейса	Русский / Английский (i18n)
Дизайн-концепция	Dark Minimalism — чёрный фон, красные акценты
3D движок	Three.js + GLTF модель гуманоидного робота
AI-интеграция	Anthropic Claude API / OpenCode-совместимые LLM

02

Цели и задачи

Основные цели:

- ❑ Создать безопасную среду для отработки клинических навыков
- ❑ Обеспечить визуальную обратную связь через 3D-модель с подсветкой симптомов
- ❑ Автоматизировать оценку правильности поставленного диагноза
- ❑ Предоставить администраторам инструменты управления контентом
- ❑ Поддерживать десятки медицинских специализаций
- ❑ Отслеживать прогресс каждого студента

Ключевые принципы UX:

- ❑ Минимализм: не более 3 основных цветов, чистая типографика
- ❑ Доступность: WCAG 2.1 AA, поддержка клавиатурной навигации
- ❑ Производительность: LCP < 2.5 с, FID < 100 мс
- ❑ Адаптивность: Desktop-first, корректное отображение на планшетах

03

Дизайн-концепция и референсы

Визуальный стиль — Dark Medical Minimalism

Интерфейс строится на принципе «меньше — больше». Тёмный фон создаёт эффект медицинского оборудования (МРТ, УЗИ-аппарат), красные акценты (#E63946) мгновенно привлекают внимание к патологиям, голубые элементы (#00B4D8) используются для навигации и информации.



Типографика:

Заголовки H1	Space Grotesk Bold / Helvetica Neue — 32–48px
Заголовки H2	Space Grotesk SemiBold — 20–28px
Body Text	Inter Regular — 14–16px, line-height 1.6
Монопространств.	JetBrains Mono — коды, диагнозы, МКБ-10
Иконки	Lucide Icons / Heroicons (outline style)

Компоненты UI (референс-описание):

- ❑ Карточки упражнений: тёмный фон (#16213E), левая граница красная 3px, hover → слабое свечение
- ❑ Кнопки: два типа — Primary (красный фон) и Ghost (прозрачный, красная обводка 1px)
- ❑ Навигация: вертикальный сайдбар 240px, иконки + текст, активный элемент с красной полоской
- ❑ Модальные окна: backdrop-blur, граница 1px полупрозрачная, без теней
- ❑ Прогресс-бары: тонкие линии (#E63946), анимированные
- ❑ Нотификации: toast снизу справа, максимум 3 одновременно
- ❑ Таблицы: зебра-полосатость #0D0D0D / #1A1A2E, без вертикальных линий

04

Архитектура системы

Технологический стек:

Слой	Технология	Обоснование
Frontend	React 18 + TypeScript + Vite	SPA, быстрый HMR, типобезопасность
3D Рендер	Three.js + React Three Fiber	Декларативный 3D в React, GLTF поддержка
Стили	Tailwind CSS + CSS Variables	Utility-first, темизация, минимальный bundle
State	Zustand + React Query	Лёгкий стейт + кэширование серверных данных
Backend	Node.js + Express / Fastify	REST API + WebSocket для реал-тайм
БД основная	PostgreSQL + Prisma ORM	Реляционные данные, типобезопасные запросы
БД сессии	Redis	JWT refresh tokens, очереди
Файлы	AWS S3 / Minio (self-hosted)	3D модели GLTF, медиа упражнений
Авторизация	JWT + Refresh Tokens + bcrypt	Безопасность, stateless
AI	Anthropic Claude API / OpenCode	Генерация сценариев, оценка диагнозов
DevOps	Docker + Nginx + GitHub Actions	CI/CD, контейнеризация

Структура директорий проекта:

```
frontend/
├─ src/
│   └─ components/
│       └─ robot/           # 3D симулятор компоненты
```

backend/

Авторизация

- ❑ Страница /login — форма email + пароль, валидация, анимированный лого
- ❑ Страница /register — регистрация студента: ФИО, университет, специализация
- ❑ Страница /forgot-password — сброс пароля по email
- ❑ JWT accessToken (15мин) + refreshToken (7 дней) в httpOnly cookie
- ❑ Защищённые роуты через НОС/middleware (поли: student, admin, superadmin)

Главная (Dashboard)

- ☐ Приветствие с именем, текущий прогресс в кольцевом графике
- ☐ Последние упражнения (3 карточки) с кнопкой продолжить
- ☐ Рекомендованные упражнения на основе слабых мест
- ☐ Статистика: пройдено/всего, средняя точность, серия дней
- ☐ Лента активности за последнюю неделю (sparkline график)

Каталог упражнений

- ☐ Сетка карточек с фильтрацией по специализации, сложности, статусу
- ☐ Поиск по названию и симптомам в реальном времени
- ☐ Каждая карточка: название, специализация, сложность (1-5★), процент прохождения
- ☐ Режим списка / сетки (переключатель)
- ☐ Пагинация или infinite scroll (решение на этапе прототипа)

Симулятор (главный экран)

- ☐ 3D робот на тёмном фоне, плавное вращение мышью (OrbitControls)
- ☐ Подсветка поражённых зон красным через material.color или ShaderMaterial
- ☐ Панель симптомов справа: список с иконками, описание каждого
- ☐ Анамнез: текстовый блок с жалобами пациента (генерируется AI)
- ☐ Панель постановки диагноза: поиск по МКБ-10, дополнительные вопросы, submit
- ☐ Результат: анимированный оверлей, правильный диагноз, объяснение, балл
- ☐ Кнопки: Пройти заново / Следующее упражнение / Вернуться в каталог

Профиль студента

- Аватар, ФИО, университет, специализация — редактируемые
- Достижения в виде бейджей (первый диагноз, 10 упражнений и т.д.)
- История прохождений с датами, баллами и правильными ответами
- Визуализация прогресса по специализациям (радарная диаграмма)

Панель администратора

- Управление пользователями: список, фильтры, смена роли, блокировка
- CRUD упражнений: создание через форму или AI-генерация
- CRUD разделов (специализаций): название, описание, иконка, порядок
- Загрузка 3D GLTF моделей и настройка зон подсветки (JSON-редактор)
- Аналитика: активность, популярные упражнения, слабые темы
- Настройки: интеграции AI, email шаблоны, лимиты

Архитектура 3D-компонента:

- Модель: GLTF/GLB гуманоид (~5–15 MB), оптимизирована Draco-компрессией
- Рендерер: WebGLRenderer (antialiasing on, shadows on), HDR окружение
- Камера: PerspectiveCamera, OrbitControls с damping, зум ограничен
- Подсветка симптомов: замена material на MeshStandardMaterial с emissiveColor #E63946
- Анатомические зоны: named meshes в GLTF (head, chest, abdomen, left_leg, right_leg и т.д.)
- Анимации: idle-дыхание (LoopRepeat morph targets), highlight pulse через THREE.Clock
- Интерактивность: Raycaster для клика по зонам тела (показывает детали симптома)
- Лоадер: прогресс-бар с процентами пока GLTF грузится
- Fallback: 2D SVG анатомический силуэт если WebGL недоступен

Структура данных симптомов (JSON):

Медицинские специализации (разделы)

Система поддерживает неограниченное количество разделов. Начальный набор включает следующие специализации:

#	Специализация	Примеры сценариев
1	Хирургия	Острый живот, аппендицит, кишечная непроходимость
2	Травматология	Переломы, вывихи, ЧМТ, политравма
3	Дерматология	Дерматиты, псориаз, инфекционные поражения кожи
4	Педиатрия	Детские инфекции, нарушения развития, острые состояния
5	Кардиология	ИБС, ОИМ, аритмии, сердечная недостаточность
6	Неврология	ОНМК, эпилепсия, нейроинфекции
7	Пульмонология	Пневмония, ХОБЛ, астма, плеврит
8	Эндокринология	Сахарный диабет, патология щитовидной железы
9	Гастроэнтерология	Язвенная болезнь, гепатиты, панкреатит
10	Нефрология	ОПН, ХБП, гломерулонефрит
11	Ревматология	Артриты, СКВ, остеопороз
12	Инфектология	ВИЧ, гепатиты, туберкулёз, сепсис
13	Онкология	Ранняя диагностика, паранеопластические синдромы
14	Психиатрия	Психозы, депрессия, деменция
15	Акушерство и гинекология	Осложнения беременности, ГСЗ
16	Офтальмология	Острые состояния, глаукома, ретинопатии
17	Оториноларингология	ЛОР-инфекции, острые кровотечения
18	Урология	МКБ, острый цистит, простатит
19	Токсикология	Острые отравления, антидотная терапия
20	Скорая помощь	Комплексные тяжёлые пациенты, триаж

08 API — Основные эндпоинты

Группа	Метод + Путь	Описание
AUTH	POST /api/auth/login	Вход, возвращает JWT
AUTH	POST /api/auth/register	Регистрация студента
AUTH	POST /api/auth/refresh	Обновление токена
AUTH	POST /api/auth/logout	Выход, очистка cookie
EXERCISES	GET /api/exercises	Список с фильтрами и пагинацией
EXERCISES	GET /api/exercises/:id	Детали упражнения + симптомы
EXERCISES	POST /api/exercises	Создать упражнение (admin)
EXERCISES	PUT /api/exercises/:id	Обновить упражнение (admin)
EXERCISES	DELETE /api/exercises/:id	Удалить упражнение (admin)
CATEGORIES	GET /api/categories	Все разделы
CATEGORIES	POST /api/categories	Создать раздел (admin)
ATTEMPTS	POST /api/attempts	Отправить диагноз
ATTEMPTS	GET /api/attempts/me	История студента
PROGRESS	GET /api/progress/me	Статистика студента
AI	POST /api/ai/generate-exercise	AI-генерация сценария (admin)
AI	POST /api/ai/evaluate	AI-оценка диагноза
USERS	GET /api/users	Список пользователей (admin)
USERS	PATCH /api/users/:id/role	Смена роли (superadmin)
MEDIA	POST /api/media/upload	Загрузка GLTF модели (admin)

Основные таблицы (Prisma Schema):

```
id          String    @id @default(uuid())
userId      String
exerciseId  String
diagnosis   String
isCorrect   Boolean
score       Int        (0-100)
timeSec     Int
aiEvalJson  Json?
user        User       @relation(...)
exercise    Exercise   @relation(...)
createdAt   DateTime   @default(now())
}
```

Фаза 1 Инфраструктура и авторизация

1–2 нед.

- ☐ Инициализация монорепо (Turborepo или просто 2 папки)
- ☐ Docker Compose: PostgreSQL + Redis + Minio
- ☐ Prisma schema + первые миграции
- ☐ Express сервер: JWT auth роуты (/login, /register, /refresh, /logout)
- ☐ React проект: Vite + TS + Tailwind + Zustand
- ☐ Страницы Login / Register с валидацией (react-hook-form + zod)
- ☐ Защищённые роуты (PrivateRoute компонент)
- ☐ GitHub Actions: lint + тесты + build check

Фаза 2 Каталог упражнений

1–2 нед.

- ☐ CRUD эндпоинты для Category и Exercise (backend)
- ☐ Страница каталога с фильтрацией, поиском, сеткой карточек
- ☐ Страница детали упражнения (пока без 3D)
- ☐ Базовая Админ-панель: таблица упражнений, форма создания/редактирования
- ☐ Загрузка файлов в Minio (Multer middleware)

Фаза 3 3D Симулятор

2–3 нед.

- ☐ Интеграция Three.js + React Three Fiber + Drei
- ☐ Загрузка GLTF модели гуманоида (бесплатный референс: Mixamo / Ready Player Me)
- ☐ Система подсветки мешей по symptomsJson (динамическая смена материалов)
- ☐ OrbitControls + idle анимация дыхания
- ☐ Raycaster: клик по телу → тултип с симптомом
- ☐ Панель симптомов и витальных показателей справа
- ☐ Fallback SVG для no-WebGL устройств

Фаза 4 Логика диагностики и AI

2 нед.

- ❑ Панель постановки диагноза: autocomplete МКБ-10 (API или локальный JSON)
- ❑ POST /api/attempts — сохранение попытки
- ❑ Базовая оценка: exact match + Levenshtein distance
- ❑ Интеграция AI (Claude API): оценка диагноза, генерация объяснений
- ❑ AI-генерация новых сценариев упражнений (admin-инструмент)
- ❑ Экран результата: анимация, балл, объяснение

Фаза 5 Профиль, прогресс, дашборд

1–2 нед.

- ❑ Дашборд: статистика, рекомендации, последние упражнения
- ❑ Страница профиля: редактирование, достижения
- ❑ Визуализация прогресса: recharts (radar chart, bar chart)
- ❑ История попыток с фильтрами
- ❑ Email уведомления (nodemailer) — опционально

Фаза 6 Шлифовка, тестирование, деплой

1–2 нед.

- ❑ Unit тесты (Vitest + Testing Library) для критичных компонентов
- ❑ E2E тесты (Playwright) — login flow + exercise flow
- ❑ Оптимизация производительности: lazy loading, code splitting
- ❑ SEO-мета, Open Graph, PWA manifest
- ❑ Nginx конфиг, SSL, env secrets
- ❑ Production деплой, мониторинг (Sentry / Posthog)
- ❑ Финальный дизайн-ревью и accessibility аудит

11

AI-интеграция (OpenCode / Claude / DeepSeek)

Сценарии использования AI:

- ❑ Оценка диагноза: AI сравнивает ответ студента с правильным, даёт детальное объяснение ошибки
- ❑ Генерация упражнений: admin вводит специализацию + сложность, AI генерирует полный сценарий
- ❑ Дополнительные вопросы: студент задаёт уточняющие вопросы пациенту — AI отвечает в роли пациента
- ❑ Адаптивная сложность: AI анализирует историю студента и рекомендует следующее упражнение

Пример системного промпта для оценки:

```
System: Ты опытный медицинский преподаватель. Оцени диагноз студента.

Данные упражнения:
- Правильный диагноз: {correctDiagnosis} (МКБ-10: {icd10Code})
- Симптомы: {symptomsDescription}
- Витальные показатели: {vitals}

Диагноз студента: {studentDiagnosis}

Верни JSON:
{
  "isCorrect": boolean,
  "score": 0-100,
  "feedback": "объяснение на русском языке",
  "keyErrors": ["ошибка 1", "ошибка 2"],
  "recommendation": "что изучить"
}
```

Совместимость с моделями:

GLM-5.1 (Zhipu)	OpenAI-совместимый API, функции поддерживаются
DeepSeek V4 Flash	Быстрый, дешёвый, JSON-режим через response_format
Kimi K2.6 (Moonshot)	Длинный контекст, подходит для сложных анамнезов
Claude Sonnet	Наилучшее качество объяснений, нативный JSON

Рекомендация: вынести AI-провайдера в ENV переменную (AI_PROVIDER=claude|deepseek|glm|kimi) и реализовать адаптер-паттерн для смены провайдера без изменения кода.

12 Требования к безопасности

- HTTPS обязателен (Let's Encrypt), HSTS заголовок
- JWT accessToken в памяти (Zustand), refreshToken — httpOnly cookie
- Все пароли хешируются bcrypt (cost factor 12)
- Rate limiting: 5 попыток входа / минута на IP (express-rate-limit + Redis)
- CORS: только разрешённые домены
- Валидация всех входящих данных (Zod на backend)
- SQL инъекции исключены через Prisma ORM (параметризованные запросы)
- XSS: React escapes by default, DOMPurify для user-generated HTML
- Загружаемые файлы: только GLTF/GLB/PNG/JPG, сканирование MIME-type
- Секреты только в .env, не в коде (GitHub Secrets для CI/CD)
- Аудит логи: все admin-действия пишутся в отдельную таблицу AuditLog

13

Нефункциональные требования

Производительность	Lighthouse Score ≥ 90 , LCP < 2.5 с, 3D загрузка < 5 с
Масштабируемость	Горизонтальное масштабирование backend через Docker Swarm
Доступность (a11y)	WCAG 2.1 AA, поддержка screen reader на ключевых страницах
Браузеры	Chrome 100+, Firefox 100+, Safari 15+, Edge 100+
Устройства	Desktop (1280px+) — primary, Tablet (768px+) — secondary
WebGL	WebGL 2.0 обязателен для 3D; fallback SVG для старых браузеров
Uptime	99.5% SLA для production окружения
Резервное копирование	Ежедневный backup PostgreSQL и S3 в отдельный bucket
Локализация	i18next: RU (primary), EN (secondary), структура готова для KZ

14

Оценка трудозатрат и сроков

Этап	Описание	Срок	Приоритет
Фаза 1	Инфраструктура + Auth	1–2 нед.	☐ Высокий
Фаза 2	Каталог упражнений	1–2 нед.	☐ Высокий
Фаза 3	3D Симулятор	2–3 нед.	☐ Высокий
Фаза 4	Диагностика + AI	2 нед.	☐ Высокий
Фаза 5	Профиль + Прогресс	1–2 нед.	☐ Средний
Фаза 6	Тесты + Деплой	1–2 нед.	☐ Средний
ИТОГО	MVP готов к показу	8–13 нед.	—

Сроки указаны для команды из 1 full-stack разработчика. При работе с AI-ассистентами (OpenCode + DeepSeek/Claude) скорость разработки увеличивается на 30–50% за счёт генерации boilerplate, тестов и документации.

Рекомендации по работе с OpenCode / AI-моделями

Стратегия промптинга при разработке:

- ❑ Передавай полный контекст: schema + текущий файл + что нужно сделать
- ❑ Проси генерировать TypeScript-типы первыми, затем реализацию
- ❑ Для 3D компонентов: всегда указывай версию Three.js (r158) и что используется React Three Fiber
- ❑ Для Tailwind: проси использовать только стандартные классы без кастомных extend-значений
- ❑ Разбивай большие задачи: сначала types → затем api-layer → затем ui-component
- ❑ Для DeepSeek/Kimi: добавляй '// Думай пошагово' для сложных алгоритмов
- ❑ GLM-5.1: хорошо работает с функциями (tool use), используй для AI-evaluation endpoint

Пример структуры задачи для OpenCode:

Контекст:

- Проект: MedSim Pro (React 18 + TypeScript + Three.js + Express)
- Файл: frontend/src/components/robot/RobotViewer.tsx
- Зависимости: @react-three/fiber, @react-three/drei, three@0.158

Задача:

Создай компонент RobotViewer который:

1. Загружает GLTF модель из пропса modelUrl
2. Принимает массив symptoms: Array<{meshName, color, severity}>
3. Окрашивает меши с совпадающими именами в нужный цвет
4. Добавляет OrbitControls и мягкое освещение (ambient + directional)
5. Показывает LoadingSpinner пока модель грузится

Верни ТОЛЬКО код компонента с TypeScript типами.

Данное ТЗ охватывает все ключевые аспекты проекта MedSim Pro. Рекомендуется начинать с Фазы 1 и двигаться последовательно, демонстрируя рабочий прототип после каждой фазы. При возникновении вопросов по реализации — уточнять в новом промпте с максимальным контекстом (код + ошибка + ожидаемый результат).